Cómo utilizamos k6 en Semáforo Digital Mass

Sí: se crea un script, se ejecuta con k6 desde la consola y se analizan sus resultados. En este proyecto k6 se ejecutó dentro de Docker, sin instalarlo directamente en Windows. Después se usaron Python y Matplotlib para preparar las tablas y los gráficos.

Qué es k6 y qué se midió

k6 es una herramienta para probar el rendimiento. Simula usuarios mediante solicitudes HTTP a la aplicación; esta prueba no abre pantallas como un navegador. Se eligió consultar los lotes críticos: **GET /api/alerts?state=ROJO&page=0**. Fue una consulta de lectura con datos ficticios, sin registrar ventas ni cambiar el stock.

El script que utilizamos

Se creó **tests/performance/alerts.k6.js**, escrito en JavaScript. Estos son fragmentos del archivo real; el script completo también incluye métricas, verificaciones y exportación de resultados.

```
scenarios: {
  alerts: {
    executor: 'constant-vus',
    vus: Number(__ENV.VUS),
    duration: '45s',
    gracefulStop: '5s'
  }
}
```

VUS significa usuarios virtuales. El número se recibe desde la consola. Cada usuario repite la consulta durante 45 segundos y espera un segundo entre consultas. Cada uno utiliza una sesión de trabajador obtenida antes de medir; el tiempo de inicio de sesión quedó fuera de la prueba.

```
const response = http.get(
  'http://web/api/alerts?state=ROJO&page=0', {
    headers: {
      Authorization: 'Bearer ' + tokens[(__VU-1) % tokens.length]
    },
    timeout: '10s'
  }
);
sleep(1);
```

Los tres escenarios

Escenario	Usuarios virtuales	Duración	Pausa por consulta
Línea base	1	45 segundos	1 segundo
Demanda normal	3	45 segundos	1 segundo
Demanda pico	10	45 segundos	1 segundo

El script comprobaba HTTP 200, lotes en estado ROJO con stock, un máximo de 25 resultados y ausencia de costos para el trabajador. La meta era menos de 1 % de fallos; el tiempo p95 debía ser menor de 500 ms en base/normal y de 1000 ms en pico. p95 es el tiempo que no supera el 95 % de las respuestas medidas.

Cómo se ejecutó y cómo se hicieron los gráficos

Se utilizó k6 2.3.0 en la red Docker local del proyecto. El comando documentado se muestra separado en líneas para facilitar su lectura. Este ejemplo ejecuta la demanda normal; para los otros casos se cambian **VUS** y **RUN_NAME**.

```
$projectRoot = (Get-Location).Path
$scripts = "$projectRoot/tests/performance"
$private = "$projectRoot/.local/performance"
$results = "$projectRoot/docs/performance-2026-11-07"
$k6Image = 'grafana/k6@sha256:' +
  '3e4186aea62e11a45560bfc51e19eeafe82fc07ce069c78aff44b7ff893d9a5e'
docker run --rm --network semaforo-final-1107_default `
  --mount "type=bind,source=$scripts,target=/scripts,readonly" `
  --mount "type=bind,source=$private,target=/private,readonly" `
  --mount "type=bind,source=$results,target=/results" `
  -e VUS=3 -e RUN_NAME=normal $k6Image `
  run --no-color /scripts/alerts.k6.js
```

Antes de repetir: iniciar el entorno Docker y preparar sesiones válidas en el archivo privado tokens.json. Las diez cuentas del experimento se desactivaron al terminar. Los tokens no están publicados en GitHub. Conviene usar otro nombre de ejecución para conservar los resultados anteriores.

Qué resultados produjo

Escenario	Intentos	HTTP 200	HTTP 429	p95 de HTTP 200
Línea base	45	45	0	15,03 ms
Normal	135	135	0	12,67 ms
Pico	450	240	210	16,02 ms

Las respuestas HTTP 429 significan “demasiadas solicitudes”. En el pico se incumplió la meta de fallos: 210 de 450 intentos fueron rechazados (46,67 %). El primer límite observado fue el control de entrada compartido; esto no demuestra saturación de MySQL. La latencia anterior incluye solo HTTP 200.



Gráfico elaborado con los resultados originales de k6. El porcentaje de fallos incluye los rechazos HTTP 429.

De las mediciones al gráfico

k6 midió tiempos y respuestas, y guardó resúmenes JSON y registros de texto. **Python y Matplotlib** leyeron esos datos para generar el gráfico; no se inventaron los valores. CPU y memoria se obtuvieron por separado con **docker stats**. Todo se ejecutó localmente el 7 de noviembre de 2026; no mide la capacidad máxima de Railway.

Para explicarlo: “Creamos un script que simulaba varios trabajadores consultando las alertas. Lo ejecutamos con k6 mediante Docker, guardamos los resultados y después hicimos las tablas y los gráficos con Python”.

Archivos en GitHub: script y comando de ejecución · registros, JSON y gráficos originales.